6_Threads

(leftover from cache lecture)

Cache coherence

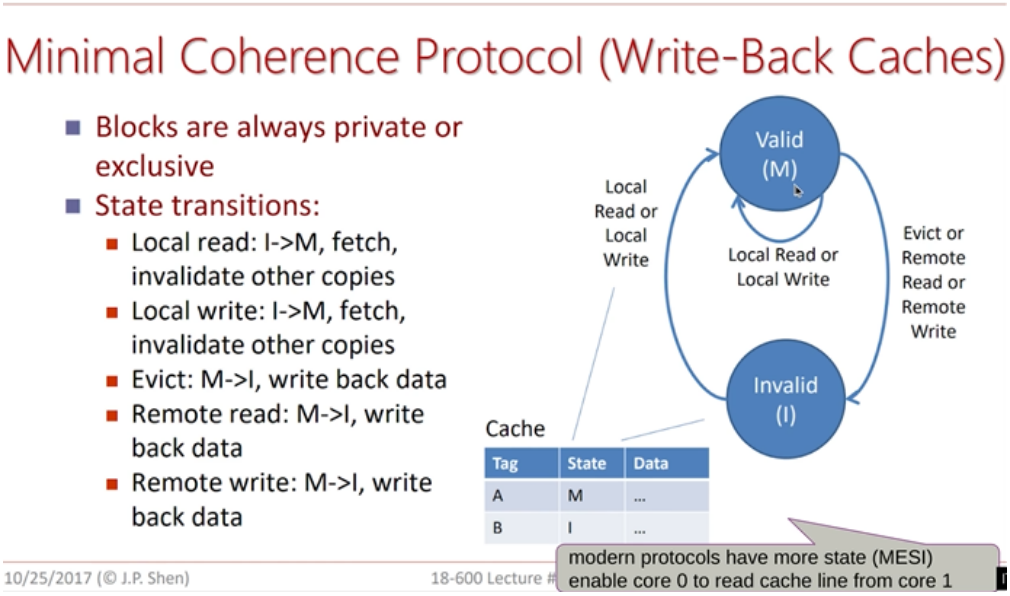
- We have multiple core and each core has a private cache
- Each one of those cores accesses main memory on a shared bus
- How to make sure the data is consistent across cores?

Solution - Cache coherence protocol

- Agreement between the cores on how to handle certain situations
- When a core writes to a variable it will notify all of the cores that if they have that variable in their cache that value is expired (invalidate that cache line)
- Then if a core tries to read this value it will not have it in its cache so it will get the new version
- There’s many kinds of cache coherence protocols

Minimal Cache coherence protocol (write-back caches)

- A cache line can be valid or invalid



- More advanced protocols can also allow cores to read from each other's caches

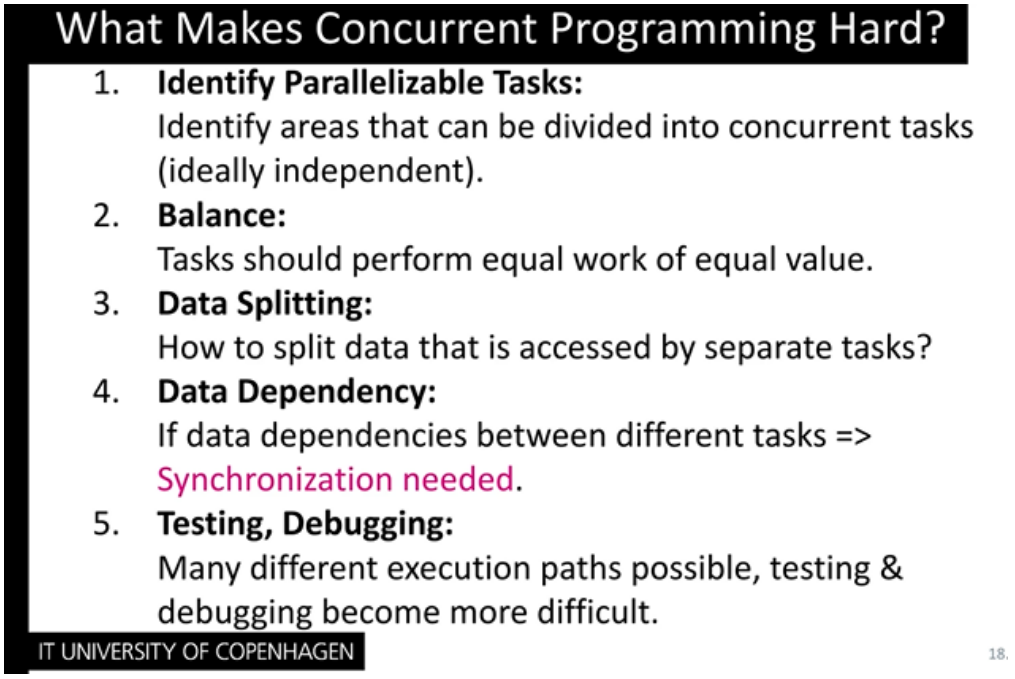
Prefetching

- Make sure that the cache lines are in the cache before you need them to avoid cache misses
- Software based
  - Explicit "fetch" instruction
  - Additional instructions executed
- Hardware based
  - Special hardware - the CPU might be built with logic that tries to predict how the program that's currently running is accessing memory
  - Can lead to unnecessary prefetching

(current lecture starts here)

Concurrent programming

- Makes our apps utilize multiple cores
- Concurrent computing - several processes are in progress at the same time (concurrently) instead of one completing before the next starts (sequentially)
    - The way this works is that a process runs for a little bit, then sleeps, the next one runs and so on in the same cycle, so progress is being made on all the processes but they're not actually all running at the same time



- Problems that can occur with concurrent programming
    - **Race condition** - multiple cores writing to the same piece of memory, which value the variable takes depends which one completes first
    - **Deadlock** - Improper resource allocation prevents progress - processes are preventing each other from completing (they are permanently stuck)
        - **Example:**
            - **Process A** holds **Resource 1** and waits for **Resource 2**, while
            - **Process B** holds **Resource 2** and waits for **Resource 1**.Neither can proceed because each process depends on the other to release a

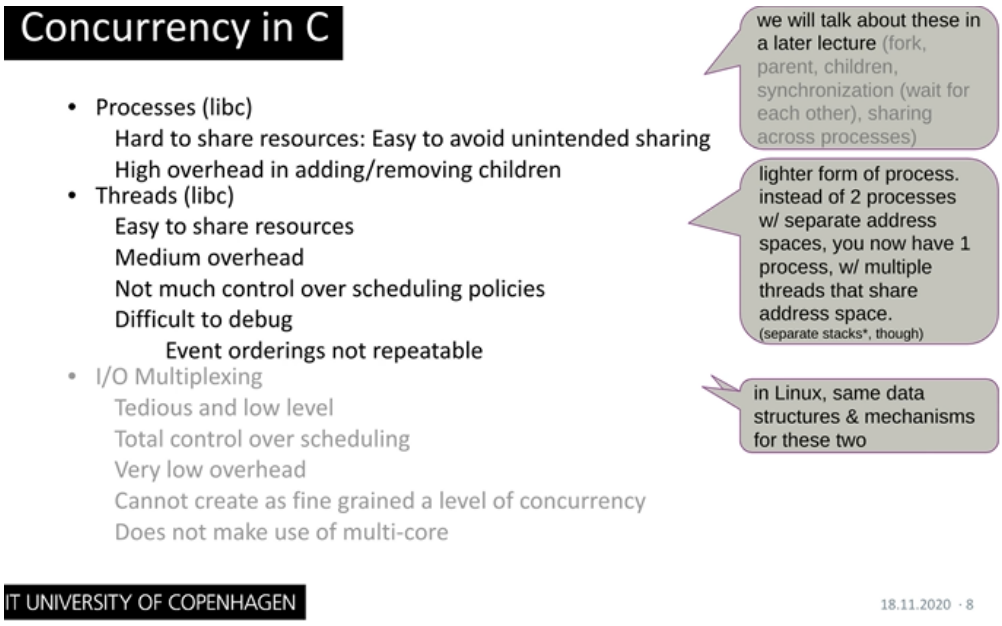
resource.

- **Livelock / Starvation / Fairness** - external events or system scheduling decisions prevent task progress
  - Happens when two or more processes are actively responding to each other, but are unable to make progress because they keep repeating the same actions
  - Unlike a deadlock, where processes are stuck waiting, in a livelock, processes are continuously **busy but ineffective**
  - **Example:**
    - Two processes detect a conflict and repeatedly back off and retry their actions (e.g., releasing and re-acquiring locks), but their actions always lead back to the same conflicting state.

Key Differences:

Aspect	Deadlock	Livelock
State	Static (processes are blocked).	Dynamic (processes actively retry but fail).
Progress	No progress—waiting indefinitely.	No progress—continual retries prevent resolution.
Cause	Circular dependency on resources.	Repeated responses to resolve a conflict.
Resolution	Needs external intervention.	May resolve spontaneously if retry strategy changes.

Strategies for concurrency in C



Threads

- A process can have many threads
- We only have one stack for the process and parts of this big stack are assigned to each individual thread
- Likewise the heap is shared among all the threads
- Threads share code, data, environment context (ex open files)
- Threads have their own register values (including PC)

How does the OS store processes?

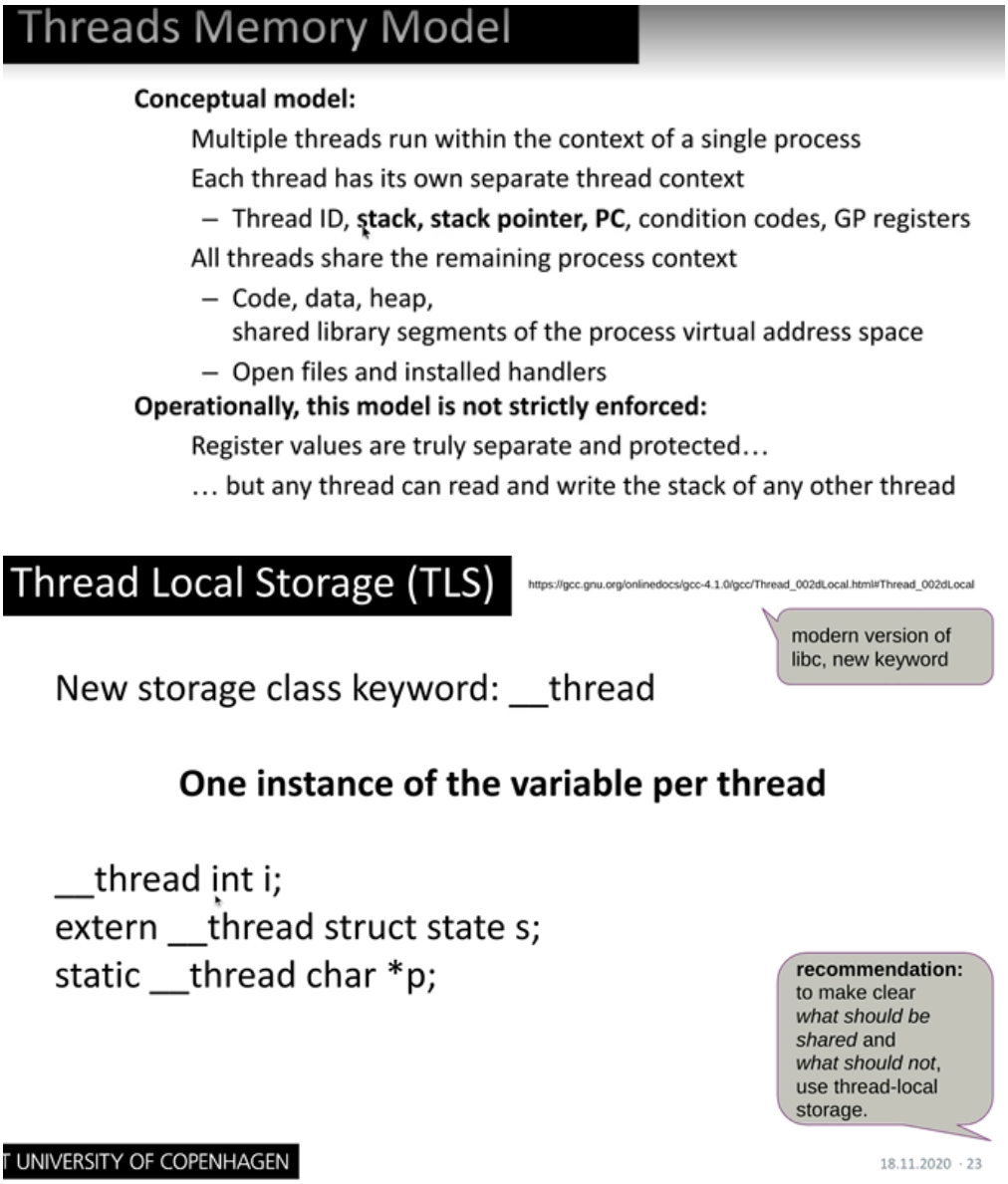
- Using a **process control block (PCB)** which includes:
  - Process ID (pid)
  - Address map (virtual memory of the process)
  - Open files
  - Pointer to parent process

- Pointer to TCBs of created threads

Threads Memory Model

Which variables are shared between threads?

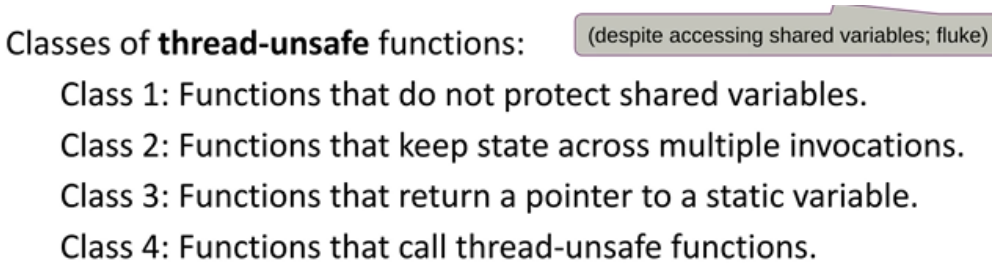
- A variable is shared if and only if multiple threads reference some instance of X



Thread safety

- **A function is thread safe if and only if it always produces correct results when called repeatedly from multiple concurrent threads (despite accessing shared variables)**

Classes of thread unsafe functions



Referential transparency

- The output of a function depends only on its input arguments and produces no side effects

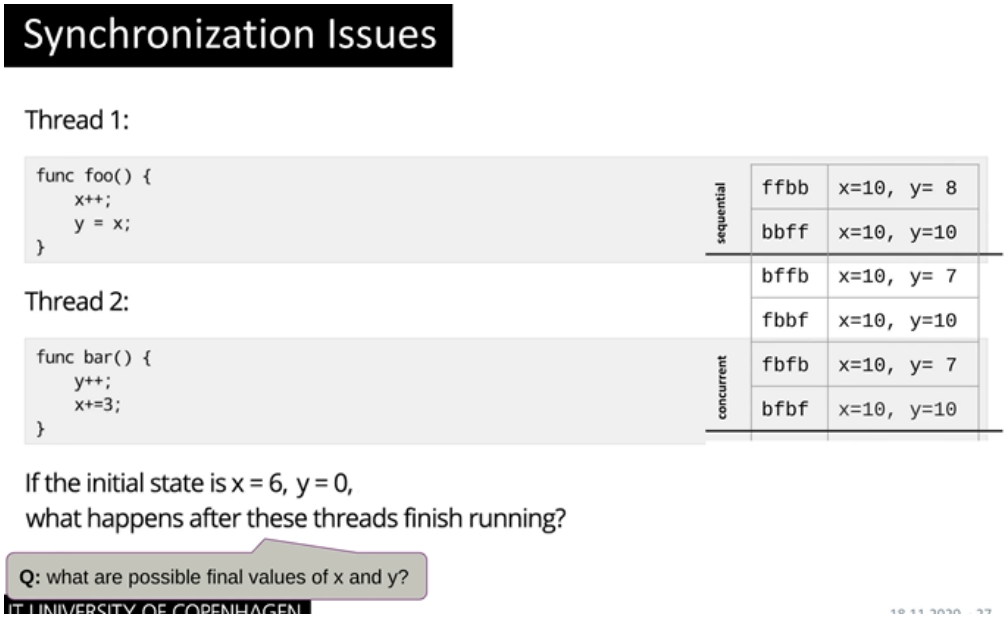
A function is referentially transparent if it can be replaced with its result without changing the program's behaviour

Reentrant functions

- A function is reentrant iff it accesses no shared variables when called by multiple threads
- This is a subset of thread-safe functions (a function being thread safe doesn't necessarily enforce it to not access shared variables)
- Require no synchronization operations

Synchronizing threads

- Multiple possible outputs for a simple code like this!



- Many operations that look like one step perform multiple operations under the hood
- When we run a multithreaded program we don't know what order threads run in or when they will be interrupted

How to synchronize threads

- Solutions vary between processors
- The below are general concepts and common cases

Bus Locking

- To avoid cores modifying each other's data you need instructions to execute atomically
- An instruction running atomically means that it won't be interrupted by other cores - it either completes fully or doesn't execute at all
    - **Characteristics of Atomic Operations**

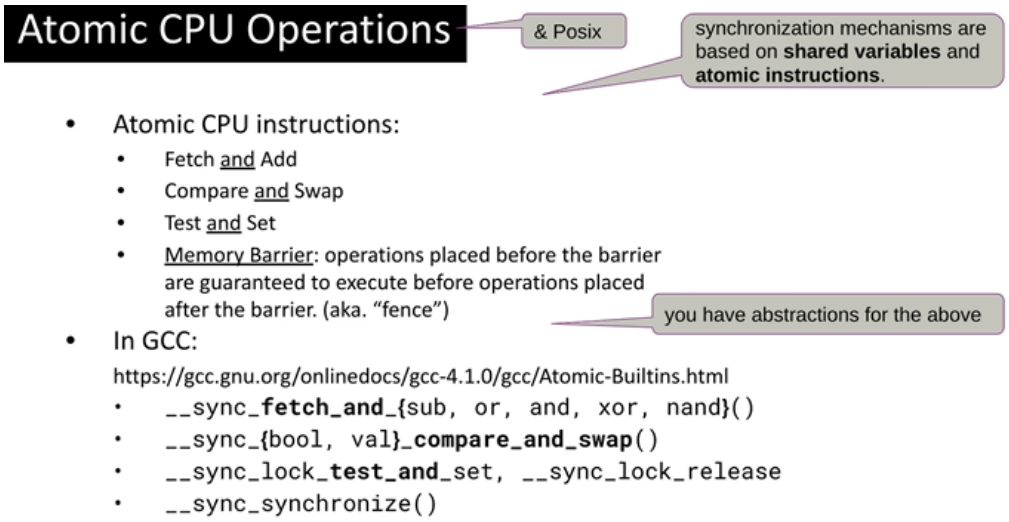
1. **Indivisibility** – The operation cannot be interrupted or split into smaller steps
2. **Consistency** – Ensures the data remains consistent before and after execution
3. **Isolation** – No other thread can observe partial changes during the execution

- In assembly - prefix instruction with the `lock` keyword
  - This makes it so that until the instruction is fully completed the other cores can't use the shared bus
- Some instructions are guaranteed to be atomic, like read and write

Weak Memory Model - instructions might be executed out of order at an assembly level - this usually happens when instructions don't depend on each other, then they might be executed in another order than specified in the program

- To prevent reordering when important you can use **memory barriers** (`volatile` keyword in C)

x86 has a strong memory model with only a little bit of reordering



Mutual exclusion (mutex)

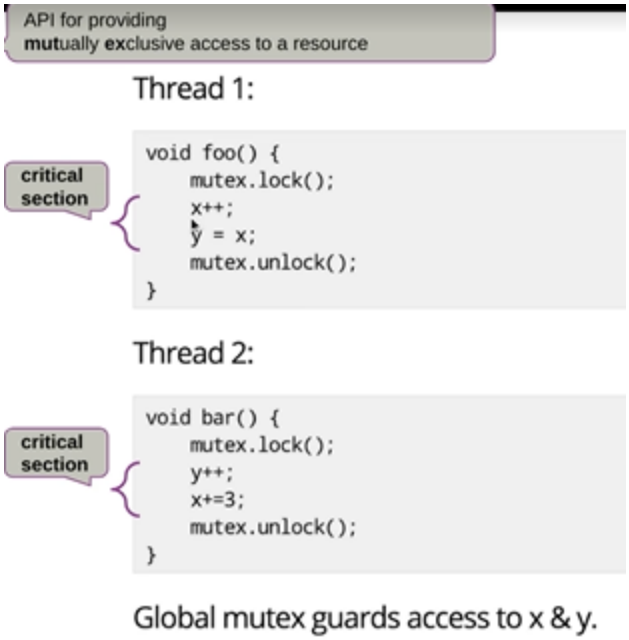
Create a mutex lock

```
mutex.lock()
```

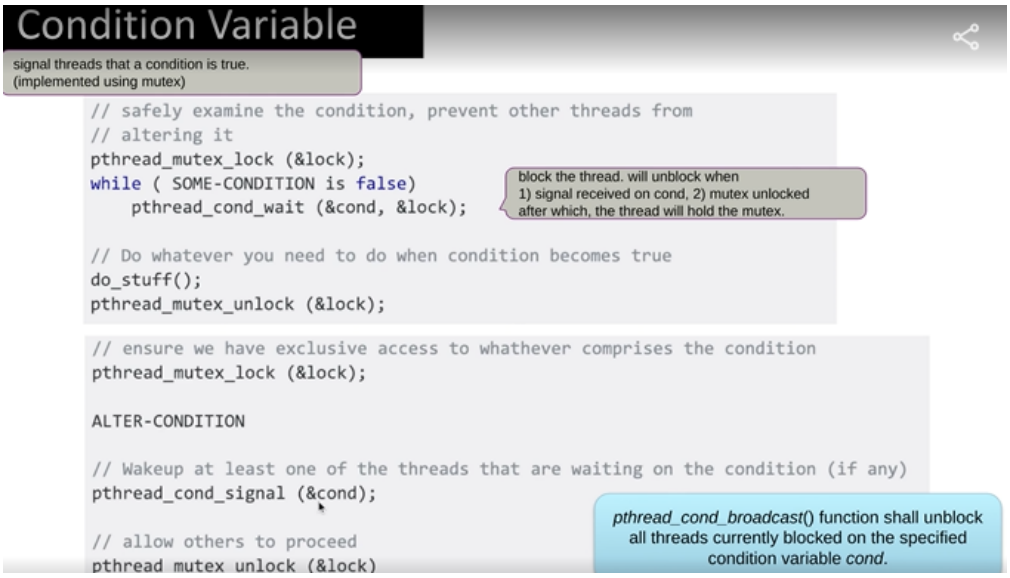
Unlock a mutex lock

```
mutex.unlock()
```

- To prevent race conditions, you need to make sure that only one thread is in a critical section where values of variables are changed



- (Can’t implement a mutex lock from scratch in code - hardware needs to be used to properly implement it)

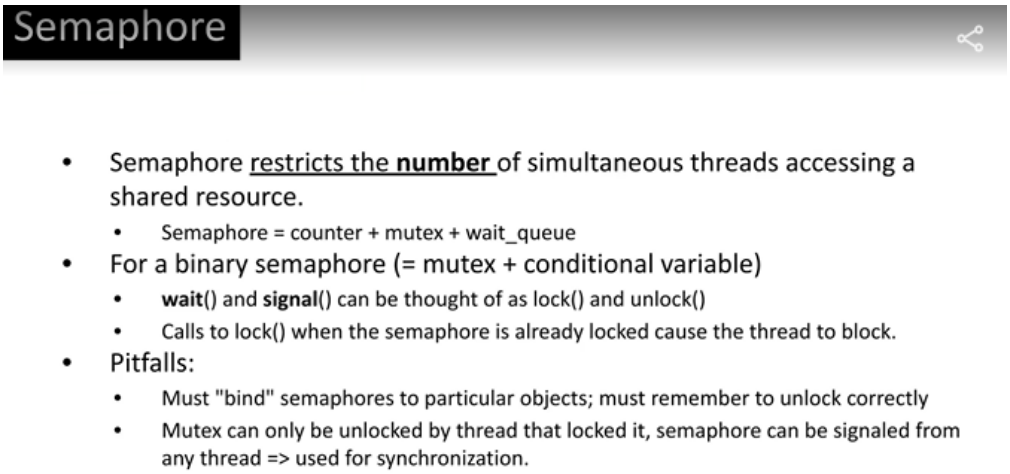


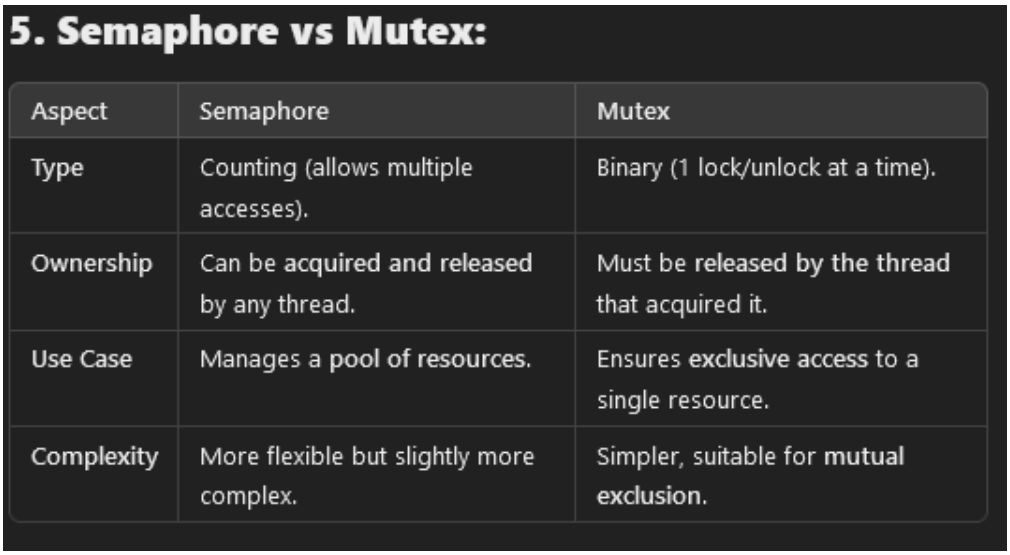
Semaphores

- Is a generalization of the mutex concept
- A flag that can be raised or lowered in one step
- It helps **coordinate and control access** by limiting the number of threads that can access a resource at the same time
- A mutex is a binary semaphore
- **Types of Semaphores:**

1. **Binary Semaphore (Mutex):**
    - Acts as a **lock** with only two states—**0 (locked)** and **1 (unlocked)**.
    - Ensures **mutual exclusion**, allowing only **one thread** to access the resource at a time.
    - Example: Protecting critical sections.
2. **Counting Semaphore:**
    - Maintains a **counter** to allow **multiple threads** (up to a set limit) to access the resource concurrently.

- Useful for managing a **pool of resources** like database connections or thread pools.





Hardware synchronization

- NVM doorbell
- Submission queue (SQ)
- Completion queue (CQ)

- Notifies storage device that the data is ready